

- ❑ Topic of the session: Advanced Java Programming
- ❑ **UNIT 6: JAVA APPLETS**
- ❑ **University of Kigali (UoK)**
- ❑ Trainer name: Dr. NTEZIRIZA NKERABAHIZI Josbert

Java Applets



- **An applet is a Java program that runs in a Web browser. An applet can be a fully functional Java application because it has the entire Java API at its disposal.**
- **An applet is a Java class that extends the `java.applet.Applet` class.**
- **A `main()` method is not invoked on an applet, and an applet class will not define `main()`.**
- **Applets are designed to be embedded within an HTML page.**

6.1 Advantage of Applet

There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured.
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

6.2 Displaying Graphics in Applet

java.awt.Graphics class provides many methods for graphics programming.

Examples

1. A "Hello, World" Applet:

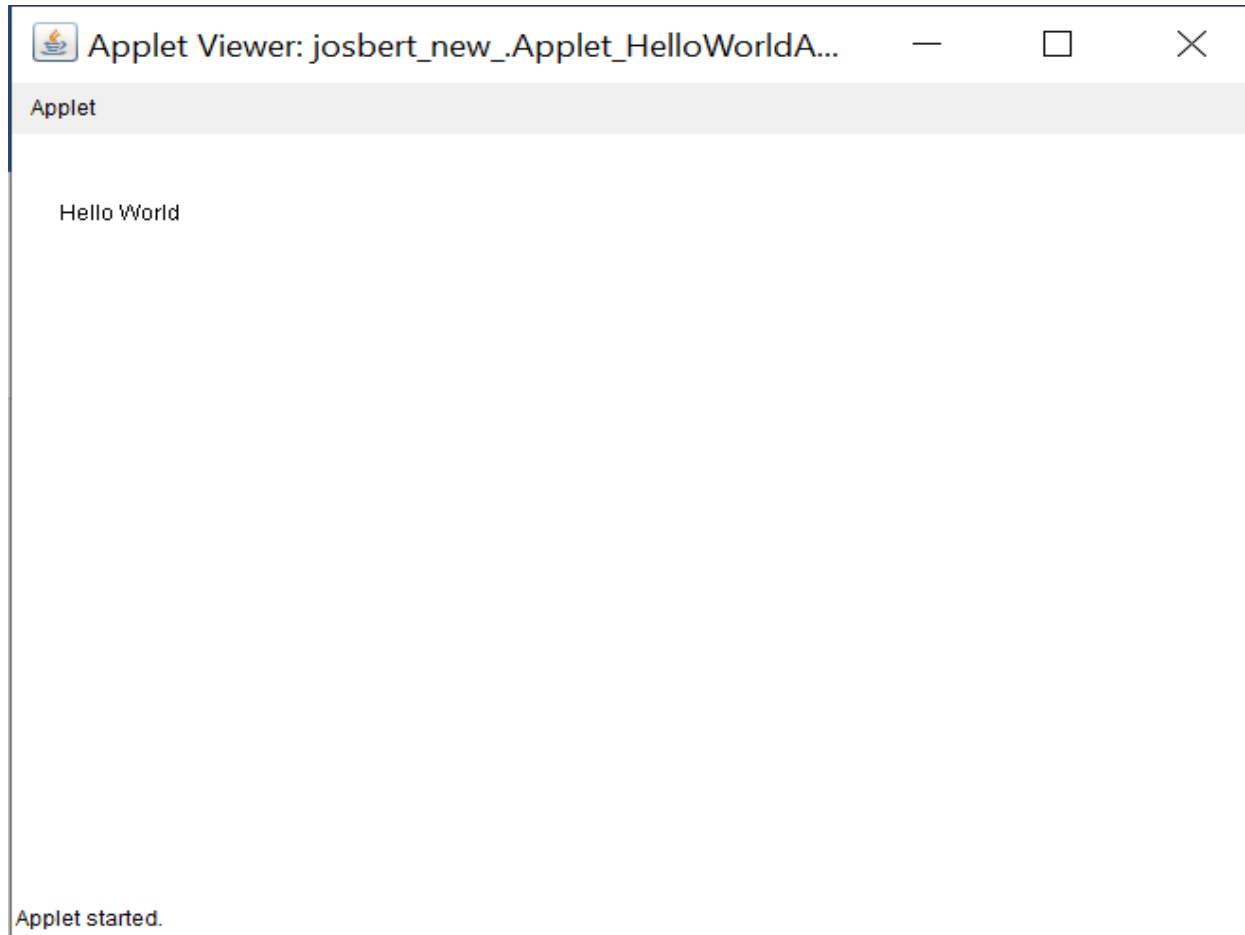
The following is a simple applet named HelloWorldApplet.java:

```
1 package josbert_new_;
2
3 import java.applet.*;
4 import java.awt.*;
5
6 public class Applet_HelloWorldApplet extends Applet {
7
8
9     public void paint (Graphics g)
10    {
11        g.drawString ("Hello World",25,50); |
12    }
13
14 }
```

UNIT 6. JAVA APPLET BASICS

6

Sample Output:



2. 1. Graphics:

These import statements bring the classes into the scope of our applet class:

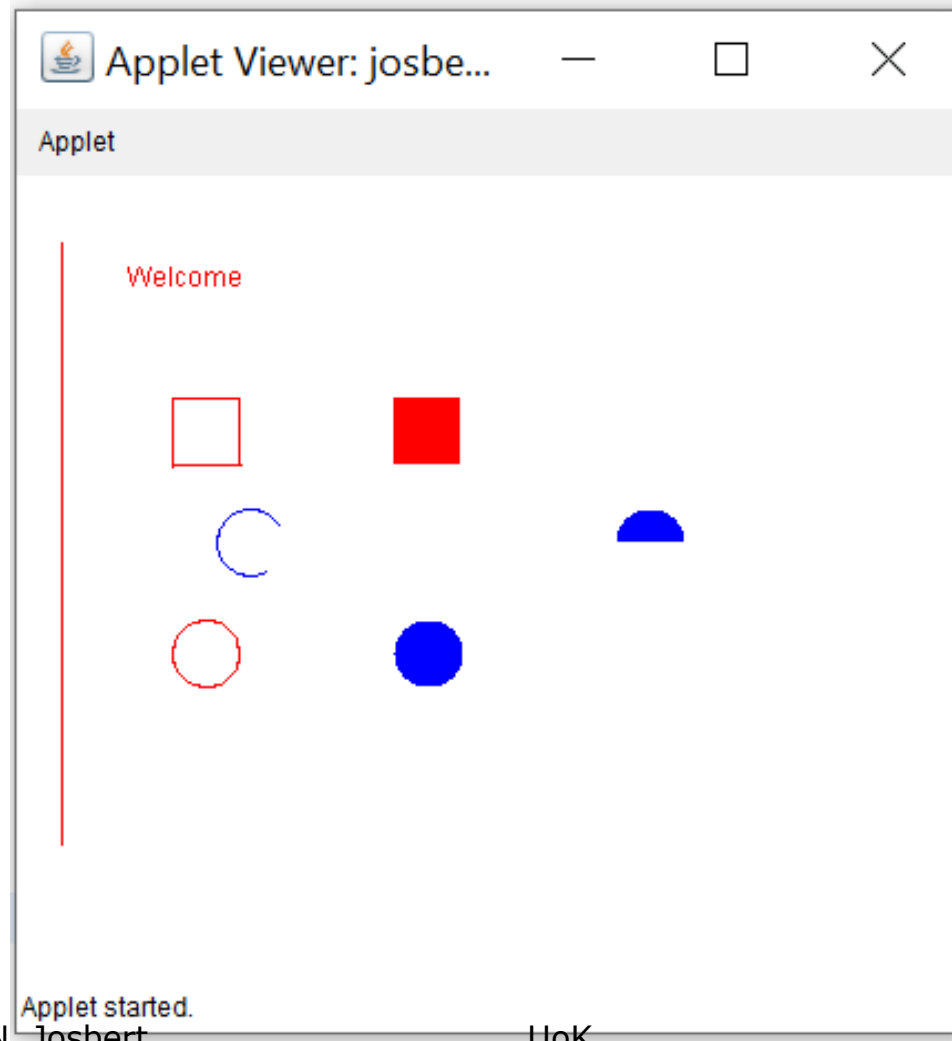
- java.applet.Applet.
- java.awt.Graphics

```
1 package josbert_new_;
2
3 import java.applet.Applet;
4 import java.awt.*;
5 public class GraphicsDemo extends Applet {
6
7     public void paint(Graphics g){
8         g.setColor(Color.red);
9         g.drawString("Welcome",50, 50);
10        g.drawLine(20,30,20,300);
11        g.drawRect(70,100,30,30);
12        g.fillRect(170,100,30,30);
13        g.drawOval(70,200,30,30);
14        g.setColor(Color.blue);
15        g.fillOval(170,200,30,30);
16        g.drawArc(90,150,30,30,30,270);
17        g.fillArc(270,150,30,30,0,180);
18    }
19
20 }
21
```

UNIT 6. JAVA APPLET BASICS

8

Sample Output:

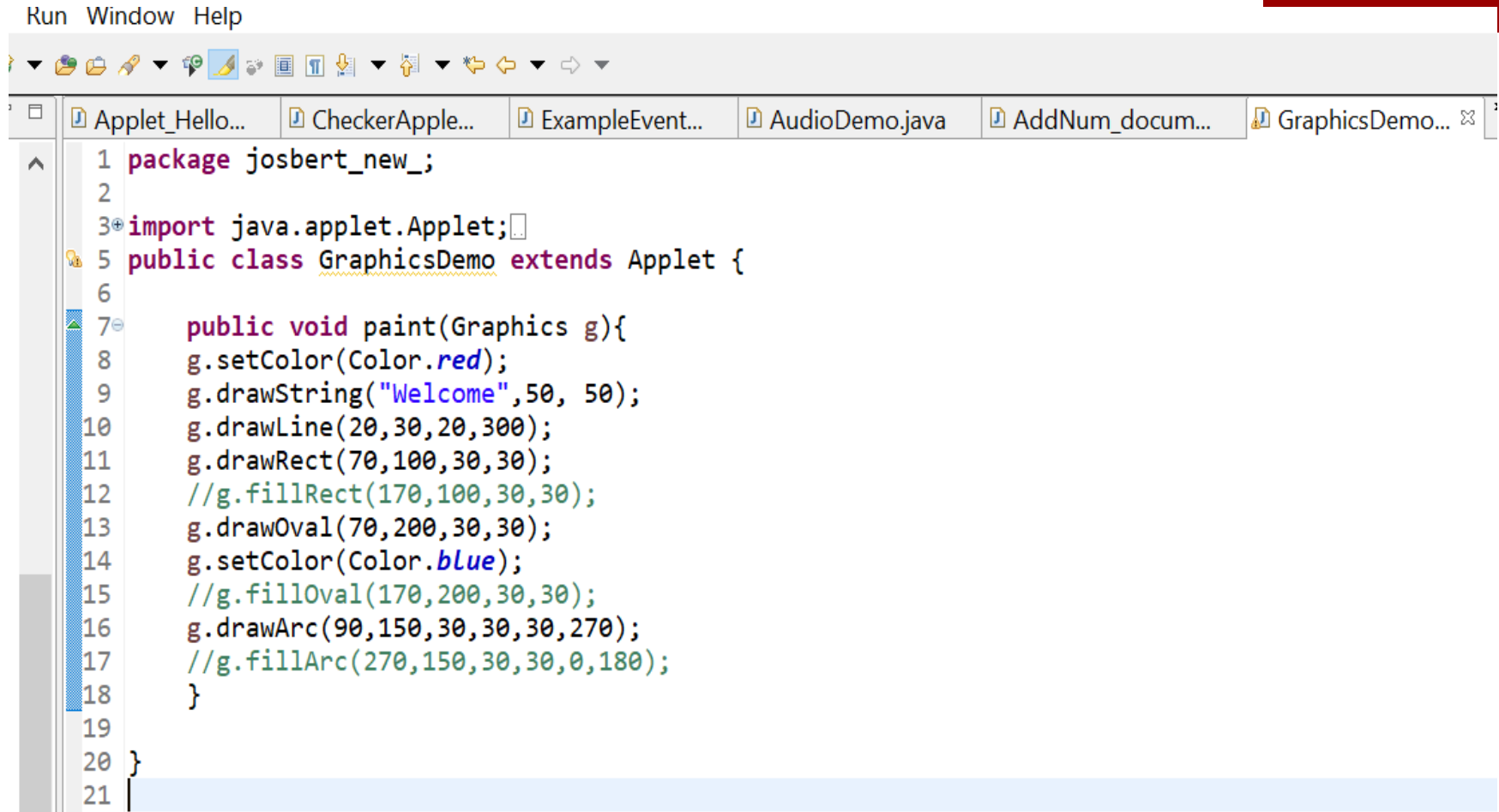


UNIT 6. JAVA APPLET BASICS

9

2. 2. Graphics:

We can remove all the fills

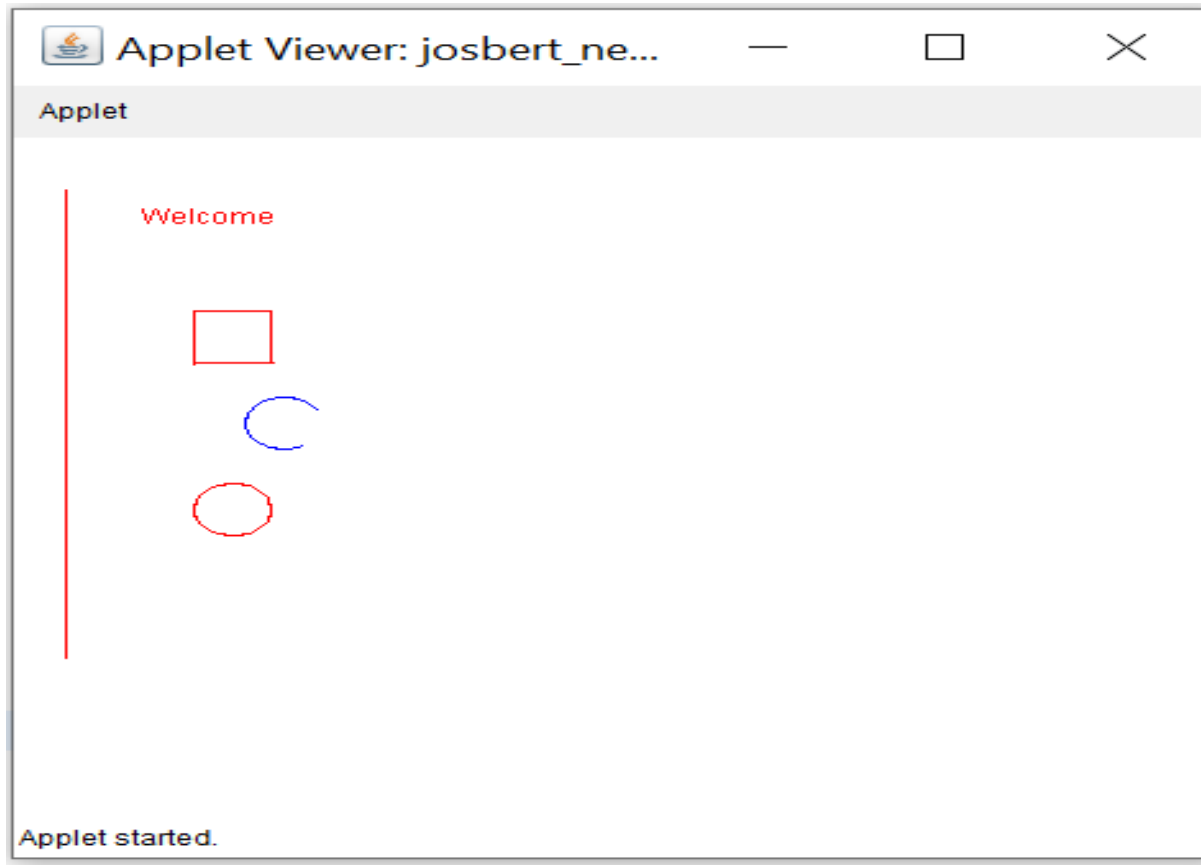


```
Run Window Help
Applet_Hello... CheckerApplet... ExampleEvent... AudioDemo.java AddNum_docum... GraphicsDemo...
1 package josbert_new_;
2
3 import java.applet.Applet;
4
5 public class GraphicsDemo extends Applet {
6
7     public void paint(Graphics g){
8         g.setColor(Color.red);
9         g.drawString("Welcome",50, 50);
10        g.drawLine(20,30,20,300);
11        g.drawRect(70,100,30,30);
12        //g.fillRect(170,100,30,30);
13        g.drawOval(70,200,30,30);
14        g.setColor(Color.blue);
15        //g.fillOval(170,200,30,30);
16        g.drawArc(90,150,30,30,30,270);
17        //g.fillArc(270,150,30,30,0,180);
18    }
19
20 }
21
```

UNIT 6. JAVA APPLET BASICS

10

Sample output:



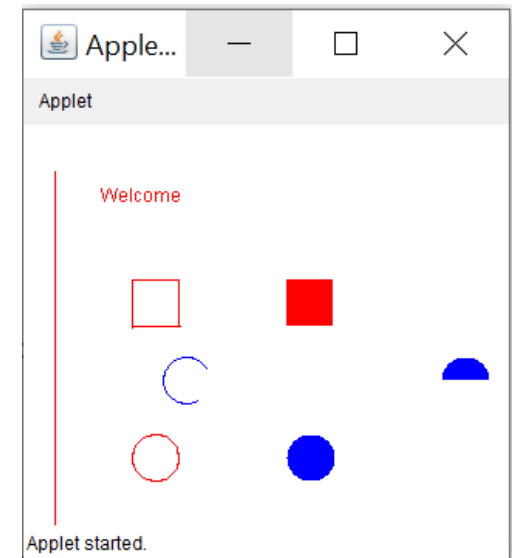
Let's add more comments to this program

```
1 package josbert_new_;
2 //Import necessary classes for creating applets and handling graphics
3 import java.applet.Applet; // Base class for applets
4 import java.awt.*; // Abstract Window Toolkit for GUI components and graphic
5
6 public class GraphicsDemo_1 extends Applet {
7
8     // Method paint() is called to perform drawing operations on the applet
9     public void paint(Graphics g){
10         // Set the current drawing color to red
11         g.setColor(Color.red);
12         // Draw a string "Welcome" at specified coordinates (50, 50)
13         g.drawString("Welcome", 50, 50);
14
15         // Draw a vertical line from (20, 30) to (20, 300)
16         g.drawLine(20, 30, 20, 300);
17
18         // Draw a rectangle with top-left corner at (70, 100)
19         //with width 30 and height 30
20         g.drawRect(70, 100, 30, 30);
21
22         // Fill a rectangle with top-left corner at (170, 100)
23         //with width 30 and height 30
24         g.fillRect(170, 100, 30, 30);
25
26         // Draw an oval defined by the rectangle with top-left corner
27         //(70, 200) with width 30 and height 30
28         g.drawOval(70, 200, 30, 30);
29
30         // Change the drawing color to blue
31         g.setColor(Color.blue);
```

Let's add more comments to this program

```
32 // Fill an oval defined by the rectangle with top-left corner
33 //(170, 200) with width 30 and height 30
34 g.fillOval(170, 200, 30, 30);
35
36 // Draw an arc defined by the rectangle with top-left corner
37 //(90, 150), width 30 and height 30,
38 // starting at 30 degrees for an extent of 270 degrees
39 g.drawArc(90, 150, 30, 30, 30, 270);
40
41 // Fill an arc defined by the rectangle with top-left corner
42 //(270, 150), width 30, height 30,
43 // starting at 0 degrees for an extent of 180 degrees
44 g.fillArc(270, 150, 30, 30, 0, 180);
45 }
46
47 }
```

Output:



15.3 EventHandlering in Applet

As we perform event handling in AWT (Abstract Window Toolkit) or Swing, we can perform it in applet also. **Let's see the simple example of event handling in applet that prints a message by click on the button.**

```
Run Window Help
CheckerApple... ExampleEvent... AudioDemo.java AddNum_docum... GraphicsDemo... EventApplet....
1 package josbert_new_;
2
3
4 import java.applet.*;
5 import java.awt.*;
6 import java.awt.event.*;
7 public class EventApplet extends Applet implements ActionListener {
8
9
10     Button b;
11     TextField tf;
12
13     public void init(){
14         tf=new TextField();
15         tf.setBounds(150,40,150,20);
16
17         b=new Button("Click");
18         b.setBounds(80,150,60,50);
19
20         add(b);add(tf);
21         b.addActionListener(this);
22
23         setLayout(null);
24     }
25
26     public void actionPerformed(ActionEvent e){
27         tf.setText("Welcome Josbert ");
28     }
29 }
30
```

Fall-2026-Dr. N. N. Josbert UoK

15.3 EventHandlering in Applet

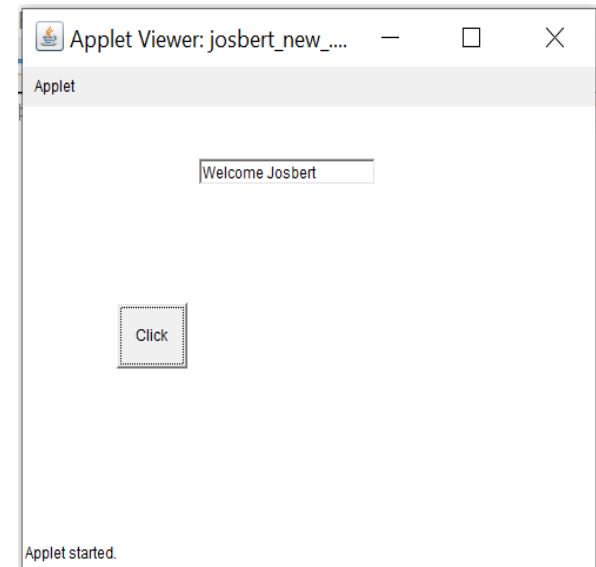
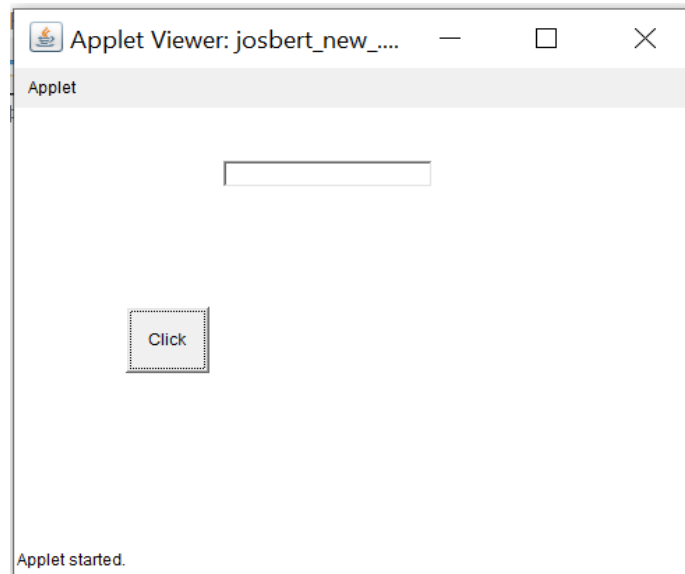
Let's add more comments to this Java program

```
1 package josbert_new_;
2
3 import java.applet.*; // Import the Applet class from the java.applet package
4 import java.awt.*; // Import the AWT package for graphical components
5 import java.awt.event.*; // Import the AWT event package for handling events
6
7 //Define the EventApplet class that extends Applet and implements ActionListener
8 public class EventApplet_cmt extends Applet implements ActionListener {
9
10     // Declare a Button and a TextField
11     Button b;
12     TextField tf;
13
14     // Initialize the applet
15     public void init() {
16         // Create a TextField object for user input
17         tf = new TextField();
18         // Set the position and size of the TextField (x, y, width, height)
19         tf.setBounds(150, 40, 150, 20);
20
21         // Create a Button object with the label "Click"
22         b = new Button("Click");
23         // Set the position and size of the Button (x, y, width, height)
24         b.setBounds(80, 150, 60, 50);
25
26         // Add the Button and TextField to the applet
27         add(b);
28         add(tf);
29         // Register the current class (EventApplet) as the ActionListener for the Button
30         b.addActionListener(this);
```

15.3 EventHandlering in Applet

```
31
32 // Set the layout manager to null so that we can use absolute positioning
33 setLayout(null);
34 }
35
36 // This method is called when an action event (like button click) occurs
37 public void actionPerformed(ActionEvent e) {
38     // Set the text of the TextField to the specified message when the button is clicked
39     tf.setText("Welcome Josbert");
40 }
41 }
42
```

Sample Output



15.3 EventHandlering in Applet

In the above example, we have created all the controls in `init()` method because it is invoked only once.

myapplet.html

1. `<html>`
2. `<body>`
3. `<applet code="EventApplet.class" width="300" height="300">`
4. `</applet>`
5. `</body>`
6. `</html>`

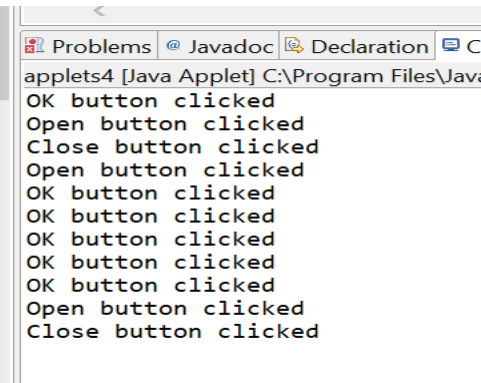
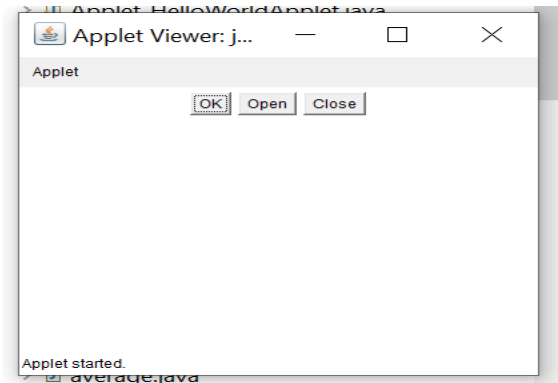
UNIT 6. JAVA APPLET BASICS

17

15.3 EventHandlering in Applet

Here's an example of creating a Java applet with buttons for OK, Open, and Close functionalities:

```
1 package josbert_new;
2 import java.applet.Applet;
3 import java.awt.Button;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6
7 public class applets4 extends Applet implements ActionListener{
8     Button okButton, openButton, closeButton;
9     public void init() {
10         okButton = new Button("OK");
11         openButton = new Button("Open");
12         closeButton = new Button("Close");
13
14         okButton.addActionListener(this);
15         openButton.addActionListener(this);
16         closeButton.addActionListener(this);
17
18         add(okButton);
19         add(openButton);
20         add(closeButton);
21     }
22
23     public void actionPerformed(ActionEvent e) {
24         if (e.getSource() == okButton) {
25             System.out.println("OK button clicked");
26         } else if (e.getSource() == openButton) {
27             System.out.println("Open button clicked");
28         } else if (e.getSource() == closeButton) {
29             System.out.println("Close button clicked");
30         }
31     }
32 }
```



6.3 EventHandlering in Applet

Let's add more comments to this Java program

```
1 package josbert_new_;
2 import java.applet.Applet; // Importing Applet class for creating the applet
3 import java.awt.Button; // Importing Button class for creating buttons
4 import java.awt.event.ActionEvent; // Importing ActionEvent class for handling button click events
5 import java.awt.event.ActionListener; // Importing ActionListener interface
6 //for defining event handling methods
7
8 public class applets4_cmts extends Applet implements ActionListener {
9     // Declaring class applets4 that extends Applet and implements ActionListener
10    // Declaring button instances for the applet
11    Button okButton, openButton, closeButton;
12
13    // Overriding the init method to initialize the applet
14    public void init() {
15        // Creating button instances with labels
16        okButton = new Button("OK");
17        openButton = new Button("Open");
18        closeButton = new Button("Close");
19
20        // Adding action listeners to each button to handle clicks
21        okButton.addActionListener(this);
22        openButton.addActionListener(this);
23        closeButton.addActionListener(this);
24
25        // Adding buttons to the applet's display
26        add(okButton);
27        add(openButton);
28        add(closeButton);
29    }
30 }
```

6.3 EventHandler in Applet

Let's add more comments to this Java program

```
31 // Overriding the actionPerformed method to define what happens on button clicks
32 public void actionPerformed(ActionEvent e) {
33     // Checking if the source of the event is the OK button
34     if (e.getSource() == okButton) {
35         // Printing to console when OK button is clicked
36         System.out.println("OK button clicked");
37     }
38     // Checking if the source of the event is the Open button
39     else if (e.getSource() == openButton) {
40         // Printing to console when Open button is clicked
41         System.out.println("Open button clicked");
42     }
43     // Checking if the source of the event is the Close button
44     else if (e.getSource() == closeButton) {
45         // Printing to console when Close button is clicked
46         System.out.println("Close button clicked");
47     }
48 }
49 }
```

6.3 EventHandlering in Applet

In this example:

- We create an applet class `ButtonApplet` that extends `Applet` and implements `ActionListener`.
- Inside the `init()` method, we create three buttons: OK, Open, and Close.
- We add action listeners to each button to handle button click events.
- In the `actionPerformed()` method, we check which button was clicked and print the corresponding message.

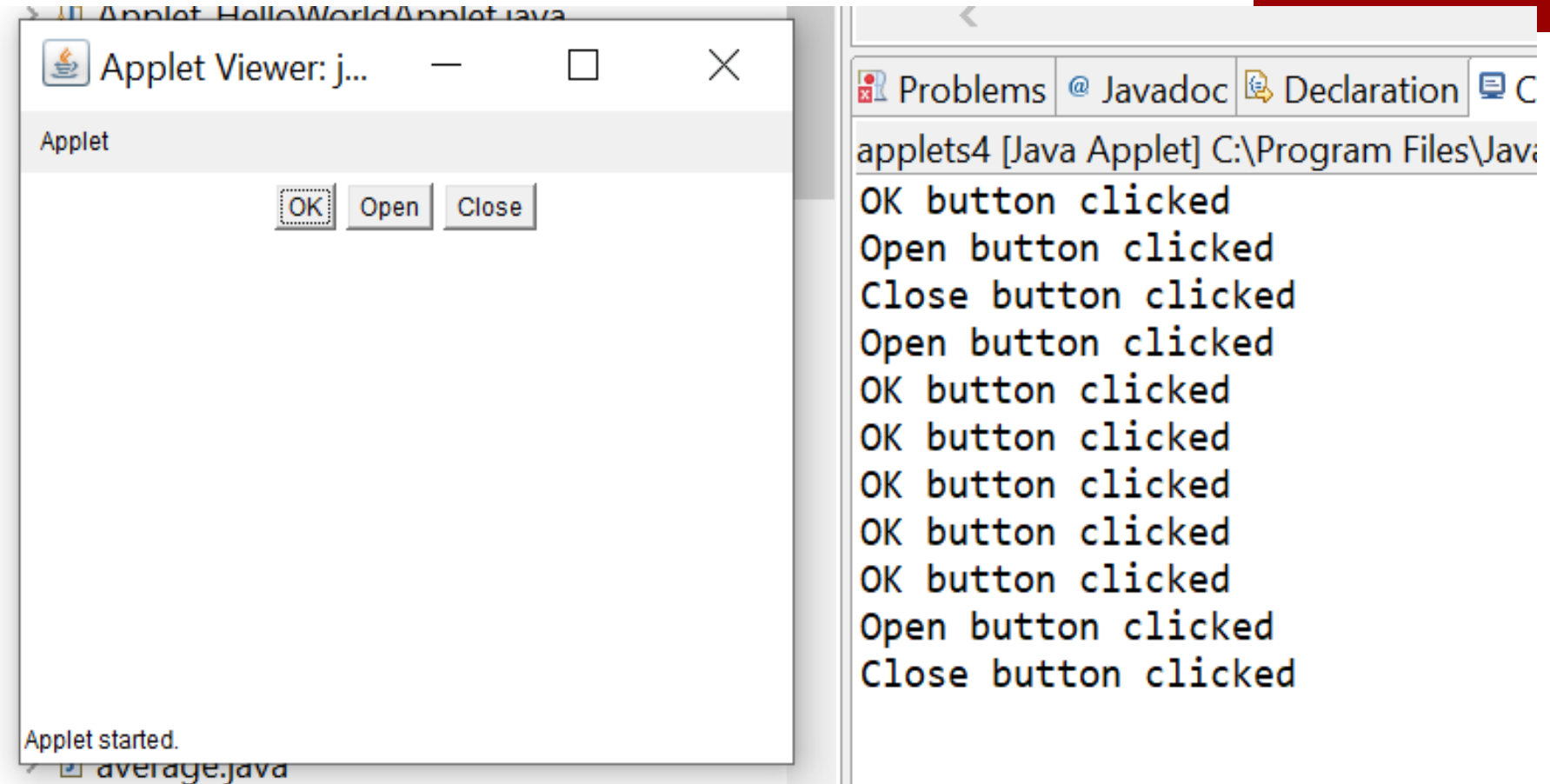
You can compile and run this code as a Java applet to see the buttons in action.

UNIT 6 JAVA APPLET BASICS

6.3 EventHandlering in Applet

Sample output:

21

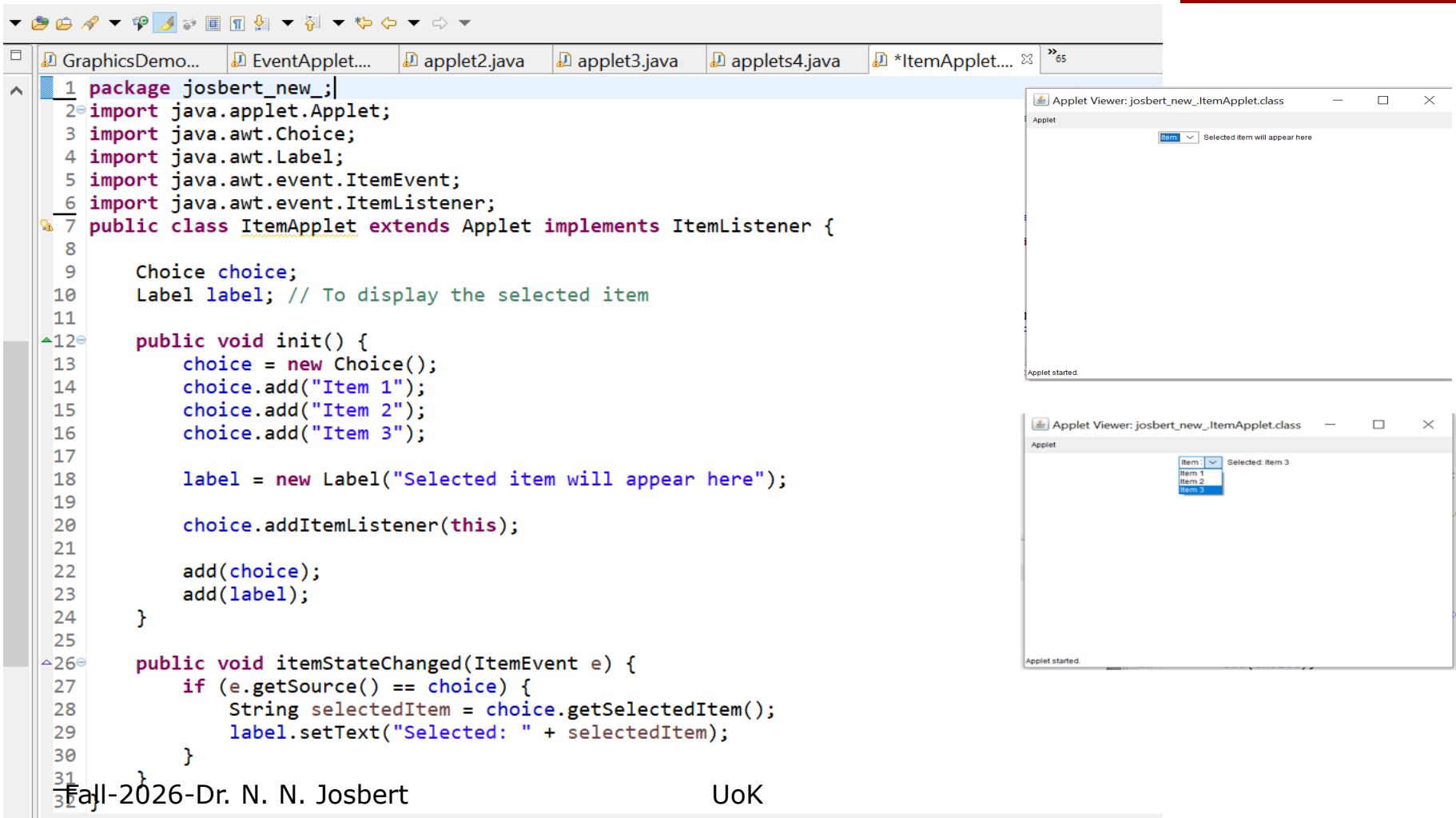


UNIT 6 JAVA APPLET BASICS

22

6.3 EventHandlering in Applet

2. Here's an example of creating a Java applet with items:



```
1 package josbert_new;|
2 import java.applet.Applet;
3 import java.awt.Choice;
4 import java.awt.Label;
5 import java.awt.event.ItemEvent;
6 import java.awt.event.ItemListener;
7 public class ItemApplet extends Applet implements ItemListener {
8
9     Choice choice;
10    Label label; // To display the selected item
11
12    public void init() {
13        choice = new Choice();
14        choice.add("Item 1");
15        choice.add("Item 2");
16        choice.add("Item 3");
17
18        label = new Label("Selected item will appear here");
19
20        choice.addItemListener(this);
21
22        add(choice);
23        add(label);
24    }
25
26    public void itemStateChanged(ItemEvent e) {
27        if (e.getSource() == choice) {
28            String selectedItem = choice.getSelectedItem();
29            label.setText("Selected: " + selectedItem);
30        }
31    }
32 }
```

The image shows a Java IDE with the source code for an applet. The code defines a class `ItemApplet` that extends `Applet` and implements `ItemListener`. It creates a `Choice` widget with three items: "Item 1", "Item 2", and "Item 3". It also creates a `Label` with the text "Selected item will appear here". The `init` method adds both the choice and the label to the applet. The `itemStateChanged` method is implemented to update the label's text whenever an item is selected from the choice.

Two screenshots of the applet's runtime behavior are shown on the right. The top screenshot shows the applet with the choice widget set to "Item" and the label displaying "Selected item will appear here". The bottom screenshot shows the applet with the choice widget expanded, showing the three items, and the label displaying "Selected: Item 3".

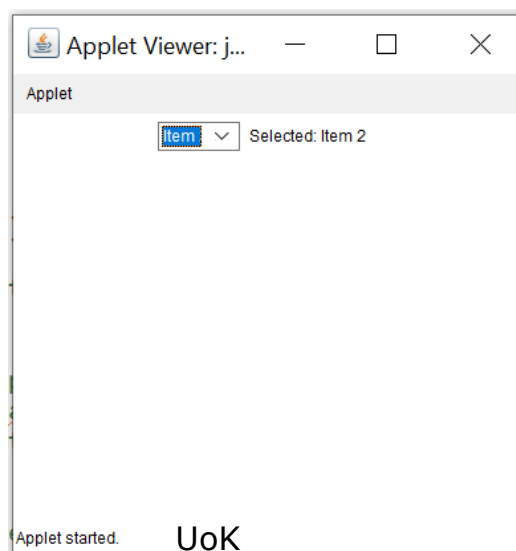
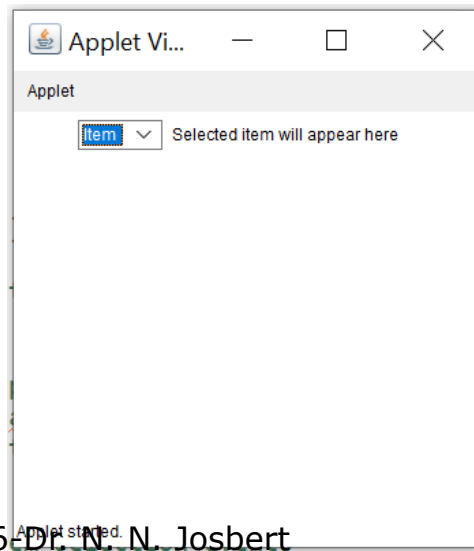
6.3 EventHandlering in Applet

Let's add more comments to this Java program

```
1 package josbert_new_;
2
3 import java.applet.Applet; // Importing the Applet class to create an applet application
4 import java.awt.Choice; // Importing the Choice class to create a drop-down menu
5 import java.awt.Label; // Importing the Label class to display text in the applet
6 import java.awt.event.ItemEvent; // Importing ItemEvent class to handle selection events
7 //from the Choice component
8 import java.awt.event.ItemListener; // Importing ItemListener interface to listen for item selection events
9
10
11 public class ItemApplet_cmts extends Applet implements ItemListener { // Declaring
12     //the ItemApplet class which extends Applet and implements ItemListener
13     // Declaring variables for the Choice component and Label to display selected item
14     Choice choice;
15     Label label; // This label will display the currently selected item from the choice menu
16
17     // Overriding the init method to set up the applet
18     public void init() {
19         // Initialize the Choice component
20         choice = new Choice();
21
22         // Adding items to the Choice component
23         choice.add("Item 1"); // Adding first item
24         choice.add("Item 2"); // Adding second item
25         choice.add("Item 3"); // Adding third item
26
27         // Initialize the Label with a default message
28         label = new Label("Selected item will appear here");
29
30         // Registering this applet as an ItemListener for the Choice component
31         choice.addItemListener(this);
```

6.3 EventHandlering in Applet

```
32
33 // Adding the Choice and Label components to the applet's display
34 add(choice); // Adding the Choice dropdown to the applet
35 add(label); // Adding the Label to show the selected item
36 }
37
38 // Overriding the itemStateChanged method to handle item selection events
39 public void itemStateChanged(ItemEvent e) {
40     // Checking if the event source is the Choice component
41     if (e.getSource() == choice) {
42         // Retrieving the selected item from the Choice component
43         String selectedItem = choice.getSelectedItem();
44
45         // Updating the label with the selected item text
46         label.setText("Selected: " + selectedItem); // Displaying the selected item in the Label
47     }
48 }
49 }
```



6.3 EventHandlering in Applet

2.

In this example:

- We create an applet class `'ItemApplet'` that extends `'Applet'` and implements `'ItemListener'`.
- Inside the `'init()'` method, we create a `'Choice'` component with three items.
- We also create a `'Label'` to display the selected item.
- An `'ItemListener'` is added to the `'Choice'` component to handle item selection events.
- In the `'itemStateChanged()'` method, we retrieve the selected item and update the `'Label'` to display it.

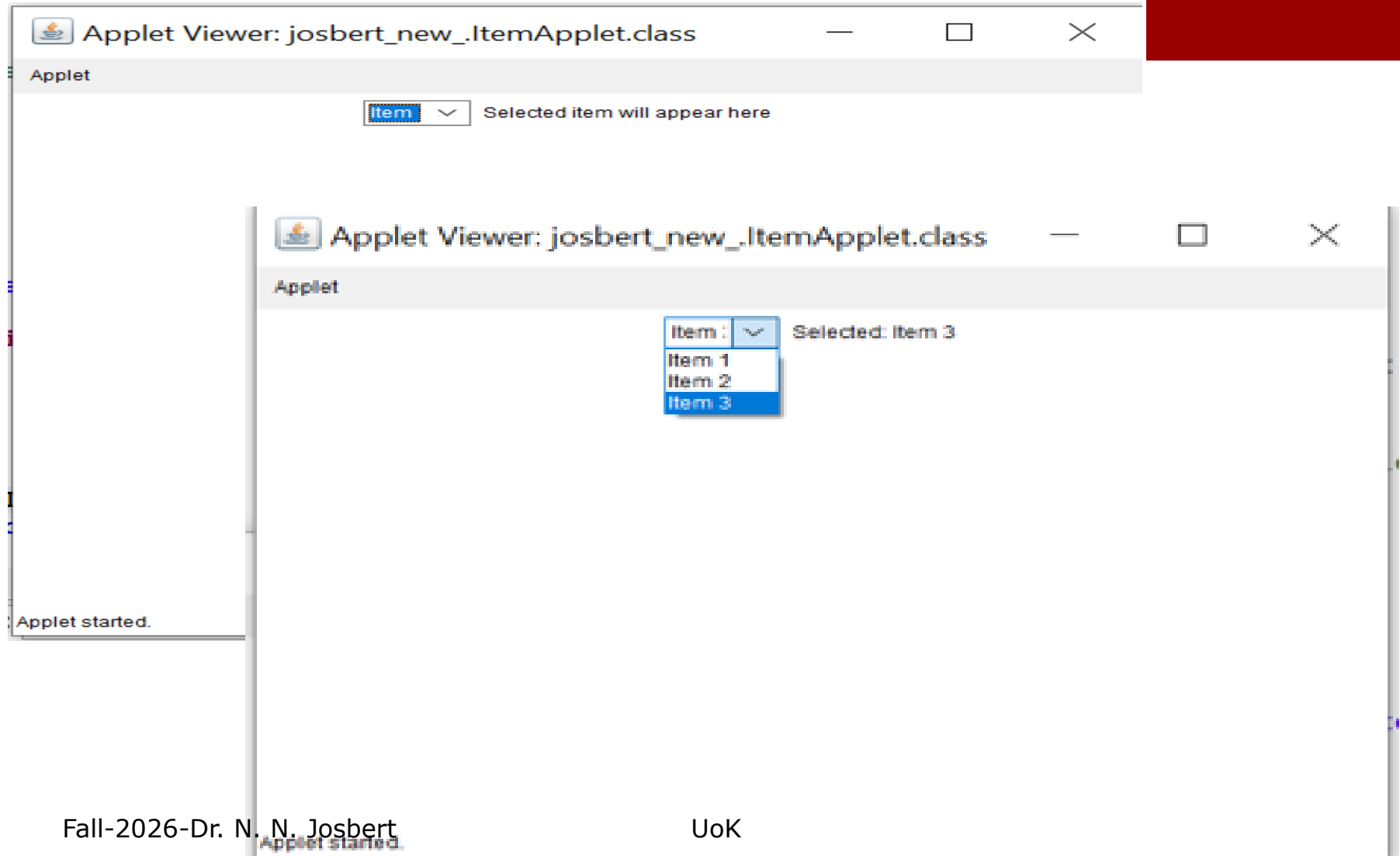
You can compile and run this code as a Java applet to see how the items in the `'Choice'` component can be selected and displayed.

UNIT 6 JAVA APPLET BASICS

26

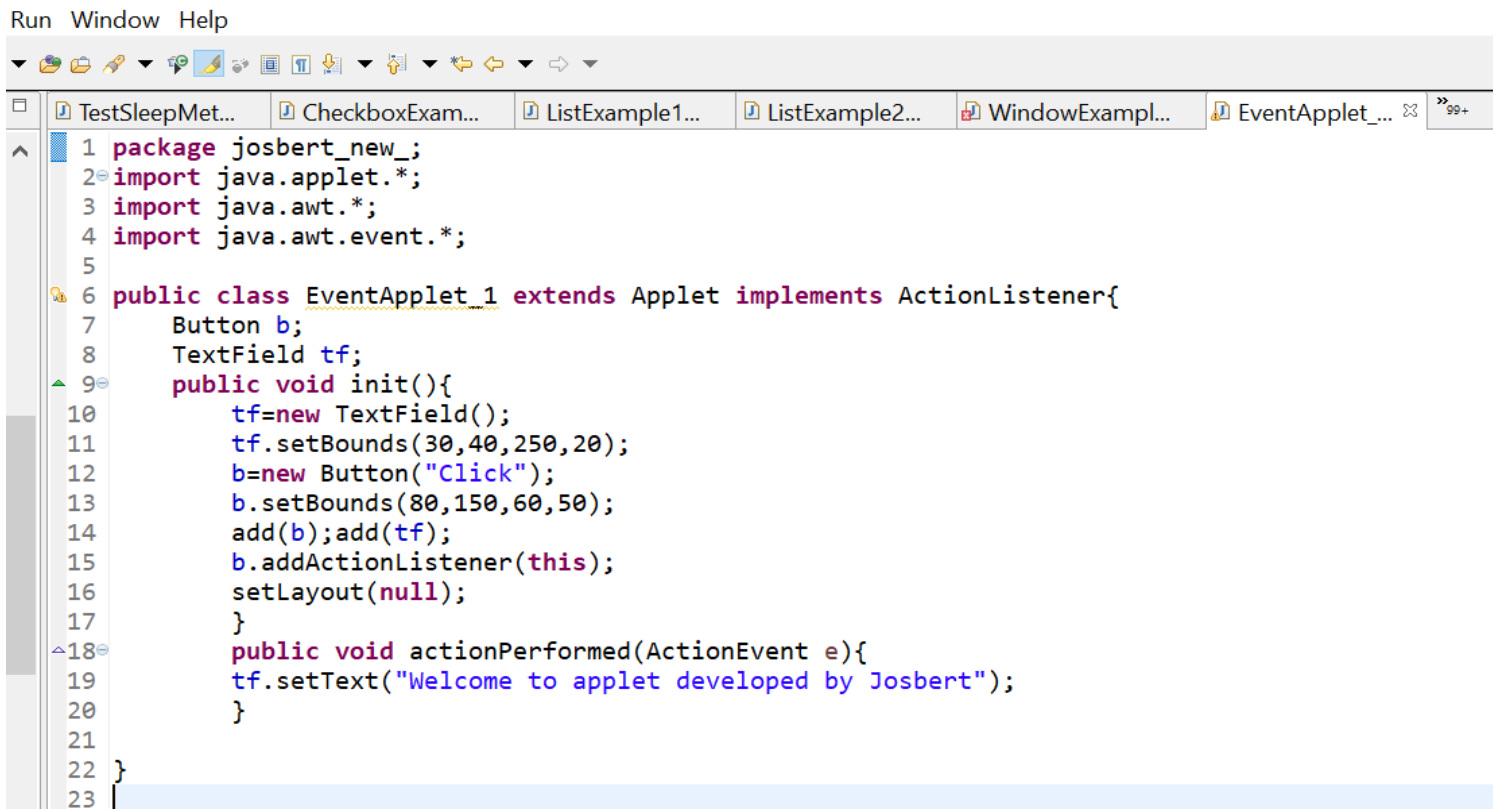
6.3 EventHandlering in Applet

Sample Output:



6.3 EventHandlering in Applet

3. As we perform event handling in AWT or Swing, we can perform it in applet also. Let's see the simple example applet that prints a message by click on the button.

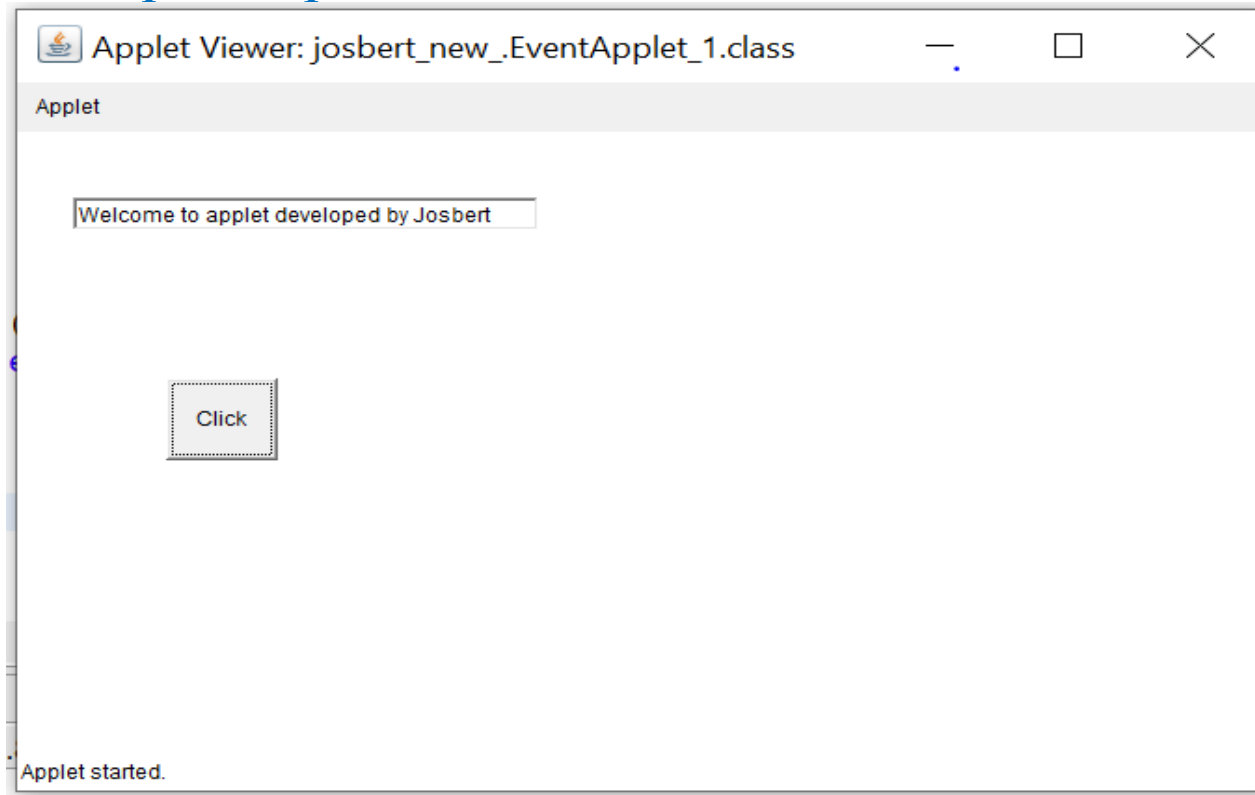


```
Run Window Help
TestSleepMet... CheckboxExam... ListExample1... ListExample2... WindowExempl... EventApplet_... >>99+
1 package josbert_new_;
2 import java.applet.*;
3 import java.awt.*;
4 import java.awt.event.*;
5
6 public class EventApplet_1 extends Applet implements ActionListener{
7     Button b;
8     TextField tf;
9     public void init(){
10         tf=new TextField();
11         tf.setBounds(30,40,250,20);
12         b=new Button("Click");
13         b.setBounds(80,150,60,50);
14         add(b);add(tf);
15         b.addActionListener(this);
16         setLayout(null);
17     }
18     public void actionPerformed(ActionEvent e){
19         tf.setText("Welcome to applet developed by Josbert");
20     }
21
22 }
23
```

UNIT 6 JAVA APPLET BASICS

6.3 EventHandlering in Applet

Sample output



UNIT 6 JAVA APPLET BASICS

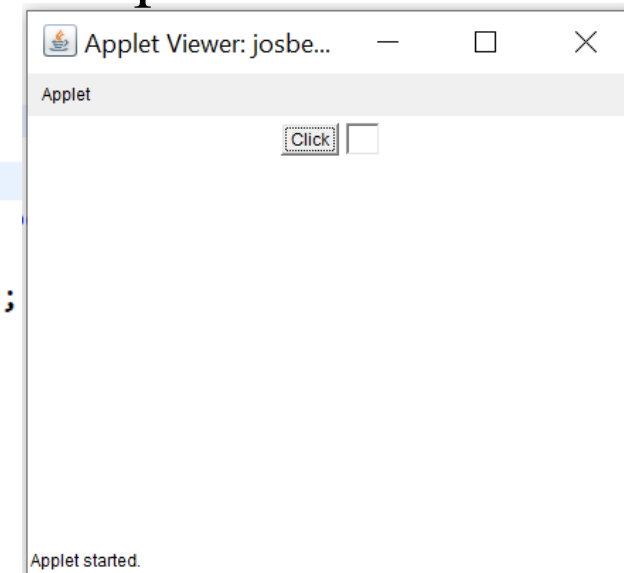
29

6.3 EventHandlering in Applet

What happens when we delete setLayout(null);

```
1 package josbert_new_;
2 import java.applet.*;
3 import java.awt.*;
4 import java.awt.event.*;
5
6 public class EventApplet_1 extends Applet implements ActionListener{
7     Button b;
8     TextField tf;
9     public void init(){
10         tf=new TextField();
11         tf.setBounds(30,40,250,20);
12         b=new Button("Click");
13         b.setBounds(80,150,60,50);
14         add(b);add(tf);
15         b.addActionListener(this);
16         //setLayout(null);
17     }
18     public void actionPerformed(ActionEvent e){
19         tf.setText("Welcome to applet developed by Josbert");
20     }
21 }
22 }
```

Output:



6.3 EventHandlering in Applet

In the above example, we have created all the controls in `init()` method because it is invoked only once.

myapplet.html

```
<html>
```

```
<body>
```

```
<applet code="EventApplet_1.class" width="300" height="300">
```

```
</applet>
```

```
</body>
```

```
</html>
```